

目录

一、实验目的	1
二、实验任务与要求	1
三、实验原理	1
1.DDS	1
2. 音乐播放器设计	4
1. 主控制模块 mcu	4
2. 乐曲读取模块 song_reader	5
3. 音符播放模块 note_player	6
4. 同步化电路	7
5. 时钟管理模块 (DCM)	8
四、设计思路及流程	8
五、主要仪器设备	8
六、实验代码及仿真结果	9
1.DDS	9
1.1verilog 代码	9
1.2 modelsim 仿真结果	10
2.mcu	10
2.1verilog 代码	10
2.2 modelsim 仿真结果	12
3.song_reader	13
3.1verilog 代码	13
3.2 modelsim 仿真结果	14
4.note_player	16
4.1verilog 代码	16
4.2 modelsim 仿真结果	17

5.sync	18
6.music_player	19
6.1verilog 代码	19
6.2modelsim 仿真结果	20
 七、vivado 工程	 20
 八、思考题	 21
 九、心得与体会	 21

浙江大学 实验报告

专业： 信息工程

姓名： _____

学号： _____

日期： 2026 年 5 月 19 日

地点： 紫金港东四 223

课程名称： 数字系统设计实验

指导老师： 屈民军，唐奕

成绩： _____

实验名称： 音乐播放器

实验类型：

同组学生姓名：

一、实验目的

1. 掌握音符产生的方法，了解 DDS 技术的应用。
2. 了解音频解码的应用。
3. 掌握系统“自顶而下”的数字系统设计方法。

二、实验任务与要求

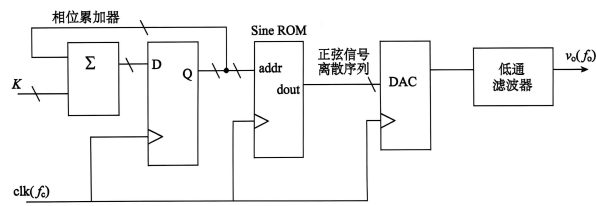
1. 设计并仿真一个 DDS 正弦信号发生器，要求：
 - (1) 采样频率 $f_c = 48\text{kHz}$ ；
 - (2) 正弦信号频率范围为 $20\text{Hz}-20\text{kHz}$ ；
 - (3) 正弦信号序列宽度 16 位，包括一位符号。
2. 设计一个音乐播放器，要求：
 - (1) 可以播放四首乐曲，并设置 play/pause_button、next_button、reset 三个按键，按下 play/pause_button 键，音乐在播放和暂停之间切换；按 next_button 播放下一首乐曲；
 - (2) LED0 指示播放情况（播放时点亮）、LED2 和 LED3 指示当前乐曲序号。

三、实验原理

1.DDS

DDS 是直接数字频率合成技术。用一个存储器（ROM / RAM）存储一张正弦表；然后将存于表中的正弦样品取出，经数模转换器 D / A，形成模拟量波形，是在数字域产生正弦

信号的方法。为了实时地改变输出信号的频率，DDS 通过改变寻址的步长来改变输出信号的频率。



图中，Sine ROM 中存放一个完整的正弦信号样品，正弦信号样品：

$$S(i) = (2^{n-1} - 1) \times \sin\left(\frac{2\pi i}{2^m}\right), \quad i = 0, 1, 2, \dots, 2^m - 1 \quad (1)$$

式中， m 为 Sine ROM 地址线位数； n 为 ROM 的数据线宽度； $S(i)$ 的数据形式为补码。

f_c 为取样时钟 clk 的频率， K 为相位增量（也称为频率控制字），输出正弦信号的频率 f_0 由 f_c 和 K 共同决定，即

$$f_0 = \frac{K \times f_c}{2^m} \quad (2)$$

正弦信号的频率 f_0 与相位增量 K 呈正比关系。为了得到更准确的正弦信号频率，相位累加器位数会增加 p 位小数。相位累加器的位数由 m 位整数和 p 位小数组成，高 m 位整数部分作为 Sine ROM 的地址。根据 DDS 输出信号的最低频率要求，可计算出 $m = 12$ ；同时相位累加器位数会增加 10 位小数。所以，相位累加器为 22 位累加器，高 12 位为 Sine ROM 的地址。

DDS 的优化设计：

因为 $m = 12$ ，所以存储一个完整周期的正弦信号样品就需要 $2^{12} \times 16 \text{ bits}$ 的 ROM。但由于正弦波形的对称性，将正弦波形分为四个区域，只需要在 Sine ROM 中存储 1/4 的正弦信号样品 (0 区) 即可。这样，Sine ROM 容量可减少为 $2^{10} \times 16 \text{ bits}$ ，即 10 位地址，存储 1/4 的正弦信号样品，共 1024 个。

优化后的框图：

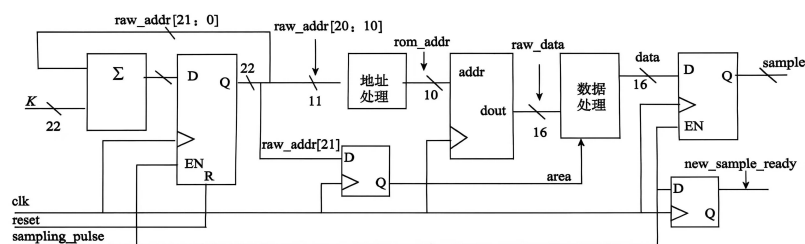


图 1: 优化后的 DDS

表 6.4 sine_rom 的地址和数据处理方法

正弦区域	Sine ROM 地址	正弦样品 data	备注
0	raw_addr[19:10]	raw_data[15:0]	
1	当 raw_addr[20:10]=1024 时, rom_addr 取 1023, 其他情况取~raw_addr[19:10]+1	raw_data[15:0]	地址镜像翻转
2	raw_addr[19:10]	~raw_data[15:0]+1	数据取反
3	当 raw_addr[20:10]=1024 时, rom_addr 取 1023, 其他情况取~raw_addr[19:10]+1	~raw_data[15:0]+1	地址镜像翻转 数据取反

图 2: sine_rom 的地址和数据处理方法

正弦查照表可用 C 或 MATLAB 程序生成。

根据实验任务可将系统划分为时钟管理模块 (DCM)、按键处理、主控制器、乐曲读取、音符播放 (note_player)、同步化电路、节拍基准产生器和音频编解码接口电路等子模块, 如图 6.44 所示。各主要子模块作用:

时钟管理模块 (DCM) 产生 100MHz 的系统时钟 sys_clk 和 12.5MHz 的音频时钟 audio_clk。主控制器 (mcu) 模块接收按键信息, 通知 song_reader 模块是否要播放 (play) 及播放哪首乐曲 (song)。

乐曲读取 (song_reader) 模块根据 mcu 模块的要求, 逐个取出音符信息 {note, duration} 送给 note_player 模块播放, 当一首乐曲播放完毕, 回复 mcu 模块乐曲播放结束信号 (song_done)。

音符播放接收到需播放的音符, 在音符的持续时间内, 以 48kHz 速率送出该音符的正弦波样品给音频编解码接口模块。当一个音符播放结束, 向 song_reader 模块发送一个 note_done 脉冲索取新的音符。

音频编解码接口模块负责将音符的正弦波样品转换为串行输出并发送给音频编解码芯片 ADAU1761。音频编解码芯片 ADAU1761 接收正弦波样品, 再进行 AD 转换并放大, 最后送至扬声器播放。注意, note_player 模块产生的正弦波样品为 16 位二进制, 需在低位加 8 个 0 后送入音频编解码接口模块。

由于音频编解码模块与系统使用不同时钟, 因此需要同步化电路协调两部分电路。节拍基准产生器产生 48Hz 的节拍定时基准脉冲信号 (beat), 而 ready 信号频率为 48kHz, 因此, 节拍基准产生器为分频比为 1000 的分频器。而按键处理模块完成输入同步化、防颤动和脉宽变换等功能。

虚框定义为一个次顶层模块 (music_player)。

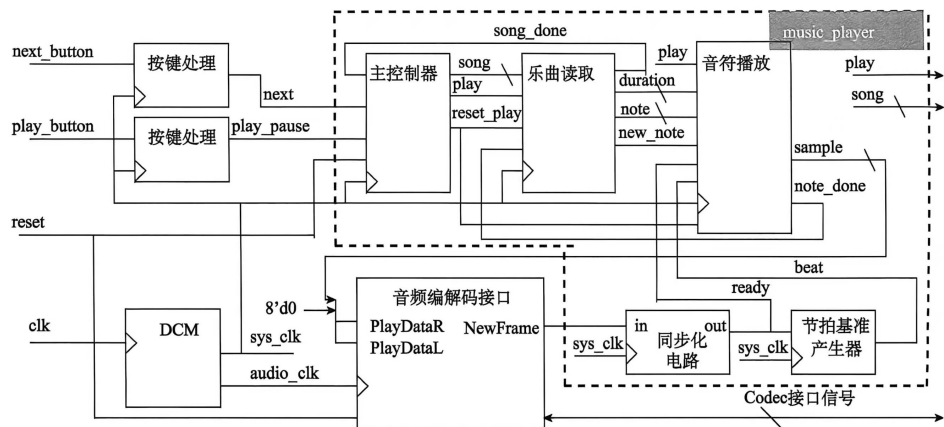


图 3: 系统顶层框图

2. 音乐播放器设计

1. 主控制模块 mcu

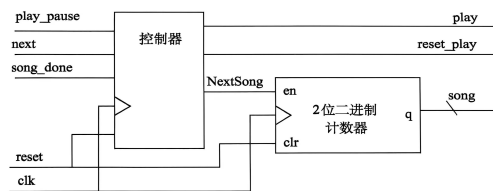


图 4: mcu 的结构框图

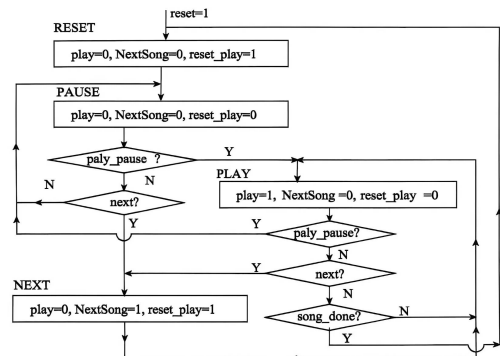


图 5: mcu 控制器的算法流程图

控制器设置初始复位（RESET）、播放（PLAY）、暂停（PAUSE）和下一首（NEXT）四种状态。系统复位后，经 RESET 状态初始化后进入 PAUSE 状态，等待各种命令输入；play pause 脉冲信号使系统在 PLAY、PAUSE 两状态之间互转；在 PLAY 或 PAUSE 状态下，若按下 next_button 按钮，则使系统进入 NEXT 状态，输出 reset_play 脉冲复位 song_reader 和 note_player 两个模块，同时输出脉冲 NextSong 乐曲序号计数器加 1，进入下一曲播放；另外，在 PLAY 状态时，若乐曲播放结束（song_done 有效）则结束播放，经 RESET 状态复位 song_reader 和 note_player 两个模块，并进入 PAUSE 状态，再次等待各种命令输入。

2. 乐曲读取模块 song_reader

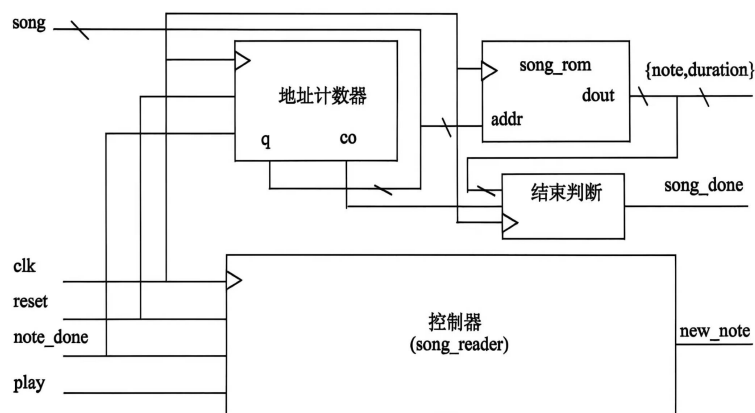


图 6: song_reader 的结构框图

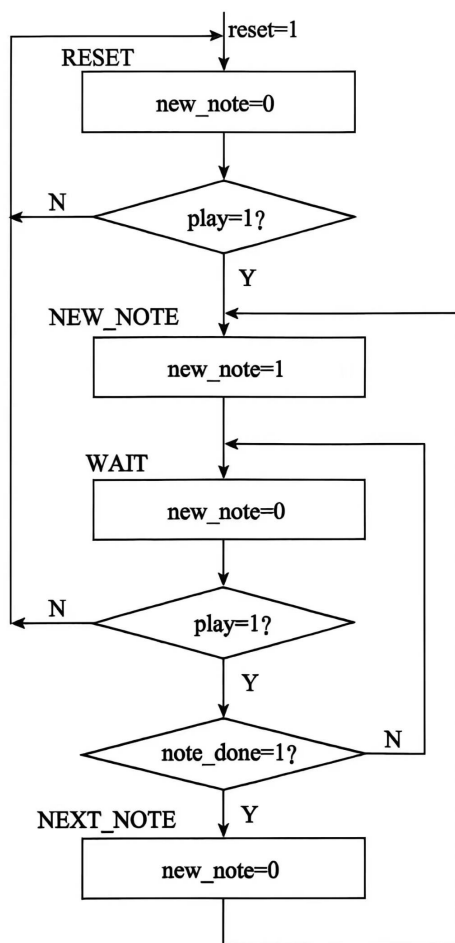


图 7: song_reader 控制器的算法流程

乐曲读取模块 song_reader 实现的功能：

- (1) 根据 mcu 模块的要求，选择播放乐曲。
- (2) 响应 note_player 模块请求，从 song_rom 中逐个取出音符 {note, duration} 送给 note_player 模块播放。
- (3) 判断乐曲是否播放完毕，若播放完毕，则回复 mcu 模块应答信号。

song_rom 是一个只读存储器，用来存放乐曲，容量为 27x12bits。共存放四首乐曲，每首乐曲占用 23x12bits 空间，即每首乐曲最长由 32 个音符组成。因此，song_rom 高 2 位地址决定哪首乐曲，而低 5 位地址决定这首乐曲的哪个音符。song_rom 每个地址存放一个音符信息，音符信息由 12 位二进制组成，高 6 位表示音符标记 note，低 6 位表示音长 duration。

根据结构框图和算法流程图，系统复位后一直在 RESET 状态等待 mcu 模块控制信号输入，当 mcu 模块发出播放命令 (play 为高电平) 时，进入 NEW_NOTE 状态输出 new_note 脉冲要求 note_player 模块播放音符；然后进入 WAIT 状态等待，当 note_player 模块播放完音符时，会发出 note_done 脉冲信号索取下一音符，note_done 脉冲信号一方面让地址计数器递增，并从 song_rom 取出一个新的音符，另一方面让控制器进入 NEW_NOTE 状态，输出 new_note 脉冲通知 note_player 模块有新的音符需要播放。当 note_done 有效，需要两个时钟周期才能从 song_rom 中读取下一个音符信息。因此新音符有效标记信号 new_note 也应在新音符数据输出后有效。在流程图中插入 NEXT_NOTE 状态，目的是延迟一个时钟周期输出信号，以配合 song_rom 的读取要求。

地址计数器为 5 位二进制计数器，其中 note_done 为计数使能输入，当 note_done 为高电平时，允许计数。计数器状态 q 为 song_rom 的低 5 位地址，song [1: 0] 为 song_rom 高两位地址。

3. 音符播放模块 note_player

音符播放模块 note_player 是本实验的核心模块，它的主要功能：

- (1) 从 song_reader 模块接收需播放的音符 {note, duration}。
- (2) 根据 note 值找出 DDS 的相位增量 k。
- (3) 以 48kHz 速率从 SineROM 取出正弦样品送给音频编解码器接口模块。
- (4) 当一个音符播放完毕，向 song_reader 模块索取新的音符。

结构框图与算法流程图：

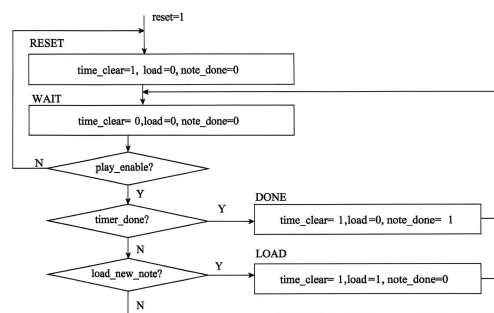
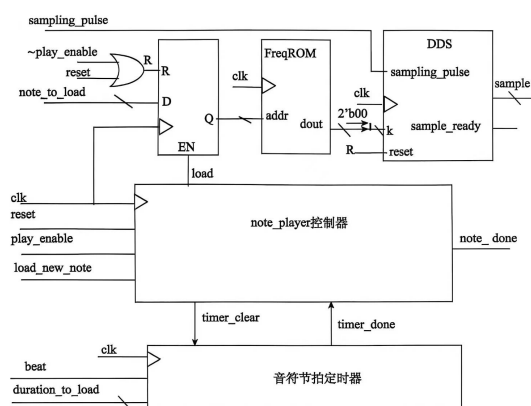


图 8: note_player 模块的结构框图

图 9: note_player 控制器的算法流程图

图中 FreqROM 为只读存储器, 完成音符标记 note 与 DDS 模块的相位增量 k 查找表关系。

note_player 控制器负责与 song_reader 模块接口, 读取音符信息, 并根据音符信息从 Frequency ROM 中读取相应相位增量 k 送给 DDS 子模块。另外, note_player 控制器还需要控制音符播放时间。在复位或未播放时, 控制器处于 RESET 状态或 PLAY 状态, 由于此时高电平 reset 或低电平 play_enable 都使 ASM 图中的 D 型寄存器清 0, 进而使 k 为 0, 不会输出正弦样品。当 play_enable 为高电平, 系统进入音符播放 PLAY 状态, 当一个音符播放结束时, 控制器进入 DONE 状态, 置位 done_with_note, 向 song_reader 模块索取新的音符, 此时 song_reader 模块输出一个 new_note 脉冲信号使控制器进入 LOAD 状态, 读取新的音符, 然后进入 PLAY 状态播放下一个音符。

音符定时器为 6 位二进制计数器，beat、timer_clear 分别为使能、清 0 信号，均为高电平有效。定时时间由音长信号 duration_to_load 决定，即 duration_to_load 个 beat 周期，timer_done 为定时结束标志。

子模块 DDS 的功能就是利用 DDS 技术产生正弦样品, DDS 模块的输入 k 为 22 位二进制, 因此需 FreqROM 输出的 20 位相位增量高位加 2 个 0 后接入 DDS。

4. 同步化电路

由于音频编解码接口模块和其他模块采用不同的时钟，因此两者之间的控制及应答信号须进行同步化处理。

本例中音频编解码接口模块的输出信号 NewFrame 的脉冲宽度为一个 audio_clk 时钟期。

需通过同步化处理，产生与 sys_clk 同步且脉冲宽度为一个 sys_clk 时钟周期的信号 read。电路由同步器和脉冲宽度变换电路组成。

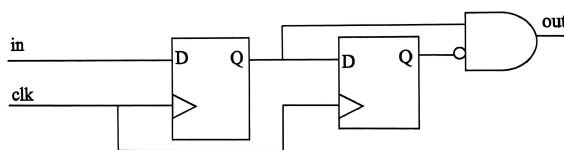


图 10: 同步化电路

5. 时钟管理模块 (DCM)

IP 内核时钟管理模块的输入时钟 clk 频率为 100MHz，产生 100MHz 的系统时钟和 12.50MHz 的音频时钟。

四、设计思路及流程

整体设计思路为先写每个控制器模块的 verilog 代码，在 modelsim 上仿真，仿真结果无误后在 vivado 上建立工程，添加相关文件，进行综合、约束、实现，下载到开发板。其中每个控制器模块又由若干子模块构成：

- 1.mcu：顶层 mcu.v、子模块控制器 mcu_control.v、D 触发器 dffr.v，counter_n.v
- 2.song_reader: 顶层 song_reader.v, 控制器 song_reader_control.v, 地址计数器 counter_n.v, 歌曲 song_rom.v, 结束判断模块 judge.v
- 3.note_player: 顶层 note_player.v, 控制器 note_player_control.v, 节拍定时器 timer.v, 只读存储器 frequency_rom.v, D 触发器 dffr.v, DDS 模块 dds.v
其中 DDS 又由顶层 dds.v, 加法器 full_adder.v, dffr.v, 地址处理模块 addr_process.v 组成
- 4.music_player :music_player.v、mcu.v、song_reader.v、note_player.v、同步化 sync.v、节拍基准产生器 counter_n.v

五、主要仪器设备

- (1) 装有 Vivado 和 ModelSim SE 软件的计算机；
- (2) Nexys Video Artix-7 FPGA 多媒体音视频智能互联开发系；

(3) 有源音箱或耳机。

六、实验代码及仿真结果

1.DDS

1.1verilog 代码

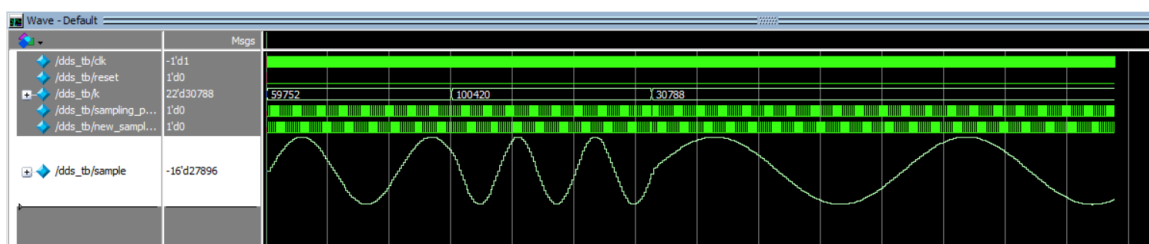
核心代码：

```
1 module dds(clk, reset, sampling_pulse, k, sample, new_sample_ready);
2     input clk, reset, sampling_pulse;
3     input [21:0] k;           //相位增量
4     output [15:0] sample;
5     output new_sample_ready;
6
7     wire [21:0] raw_addr;
8     wire [21:0] temp;
9     wire [9:0] rom_addr;
10    wire [15:0] raw_data;
11    wire [15:0] data;
12    wire area;
13
14    //加法器实现 temp = k + raw_addr
15    full_adder#(.n(22)) adder_inst(.a(k), .b(raw_addr), .s(temp), .co(),
16        .ci(0));
17
18    dffr#(.n(22)) dffr1(.d(temp), .r(reset), .en(sampling_pulse), .clk(
19        clk), .q(raw_addr));
20
21    //地址处理
22    addr_process addr_process_inst(.addr(raw_addr[20:10]), .rom_addr(
23        rom_addr));
24
25    //ROM输出原始数据
26    sine_rom sine_rom1(.clk(clk), .addr(rom_addr), .dout(raw_data));
27
28    dffr#(.n(1)) dffr2(.r(0), .en(1), .clk(clk), .d(raw_addr[21]), .q(
29        area));
```

```
26
27 assign data[15:0] = area? (~raw_data[15:0]+1) : raw_data[15:0];
28
29 dffr#(.n(16)) dff3(.r(0), .en(sampling_pulse), .clk(clk), .d(data), .
    q(sample));
30
31 dffr#(.n(1)) dff4(.r(0), .en(1), .clk(clk), .d(sampling_pulse), .q(
    new_sample_ready));
32
33 endmodule
```

Listing 1: DDS

1.2 modelsim 仿真结果



k 较大时，正弦波频率较大，k 值变化的转折点是频率变化转折点，说明这部分设计符合要求。

2.mcu

2.1verilog 代码

核心代码：

```
1 wire NextSong; // 内部信号，用于触发下一首歌曲
2
3 // 实例化计数器模块以跟踪当前歌曲
4 counter_n #(.n(4), .counter_bits(2)) counter1(
5     .clk(clk),
6     .en(NextSong),
7     .r(reset),
8     .q(song),
```

```
9         .co()
10     );
11
12     // 实例化控制模块以管理播放/暂停、下一首和复位信号
13     mcu_control mcu_control_inst(
14         .clk(clk),
15         .reset(reset),
16         .play_pause(play_pause),
17         .next(next),
18         .song_done(song_done),
19         .NextSong(NextSong),
20         .reset_play(reset_play),
21         .play(play)
22     );
```

Listing 2: mcu 顶层

```
1 always @(*)
2     begin
3         // 默认输出
4         play = 0;
5         NextSong = 0;
6         reset_play = 0;
7
8         case(state)
9             RESET: begin
10                 reset_play = 1;
11                 nextstate = PAUSE;
12             end
13             PAUSE: begin
14                 if(play_pause) nextstate = PLAY;
15                 else if(next) nextstate = NEXT;
16                 else nextstate = PAUSE;
17             end
18             PLAY: begin
19                 play = 1;
20                 if(play_pause) nextstate = PAUSE;
21                 else if(next) nextstate = NEXT;
22                 else if(song_done) nextstate = RESET;
23                 else nextstate = PLAY;
```

24
25
26
27
28
29
30
31
32

Listing 3: mcu_control 控制模块

2.2 modelsim 仿真结果

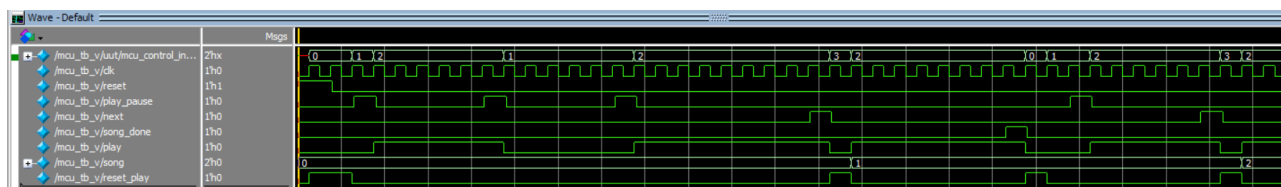
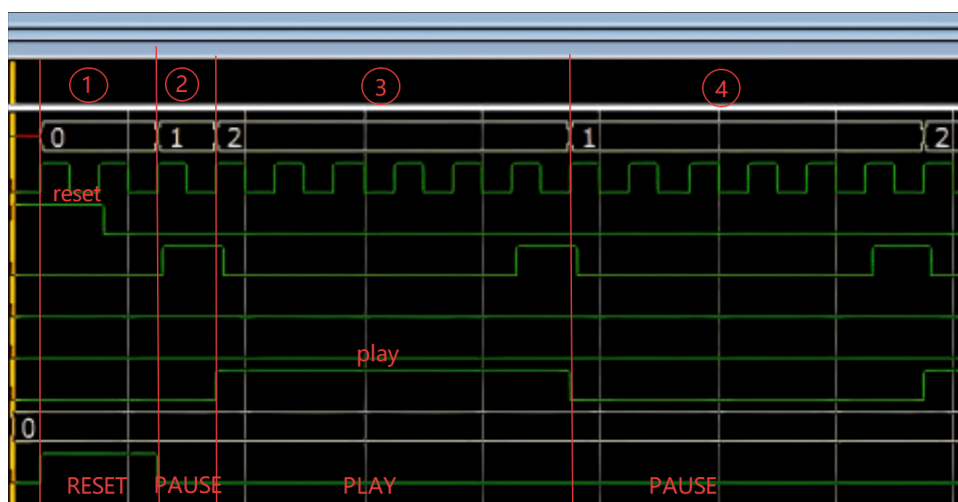


图 11: mcu 仿真图

放大:



区域 1 中, 时钟上升沿到来前 $\text{reset} = 1$, 时钟上升沿到来时 $\text{state} = \text{RESET}$, 输出 $\text{play} = 0$, $\text{nextsong} = 0$, $\text{reset_play} = 1$.

区域 2 中, 时钟上升沿到来前 $\text{reset} = 0$, 时钟上升沿到来时, state 变为 PAUSE, 输出 $\text{play} = 0$, $\text{nextsong} = 0$, $\text{reset_play} = 0$.

区域 3 中，时钟上升沿到来前 $\text{reset} = 0$, $\text{play_pause} = 1$, play 按钮按下, 时钟上升沿到来时 $\text{state} = \text{PLAY}$, 输出 $\text{play} = 1$, $\text{nextsong} = 0$, $\text{reset_play} = 0$.

区域 4 中，时钟上升沿到来前 $\text{reset} = 0$, $\text{play_pause} = 1$, $\text{play} = 1$, pause 按钮按下, 时钟上升沿到来时, $\text{state} = \text{PAUSE}$, 输出 $\text{play} = 0$, $\text{nextsong} = 0$, $\text{reset_play} = 0$.

经上述分析，mcu 模块的设计符合要求。

3.song_reader

3.1verilog 代码

核心代码：

```
1 module song_reader(clk, reset, note_done, play, song, song_done, new_note
  , duration, note);
2   input clk, reset, note_done, play;
3   input [1:0] song;
4   output song_done, new_note;
5   output [5:0] duration, note;
6
7   wire co;
8   wire [4:0] q;
9
10  // 5 位计数器，模为 32
11  counter_n #(.n(32), .counter_bits(5)) conter_inst2(.clk(clk), .en(
    note_done), .r(reset), .q(q), .co(co));
12  // 歌曲读取的控制逻辑
13  song_reader_control song_reader_control_inst(.clk(clk), .reset(reset)
    , .note_done(note_done), .play(play), .new_note(new_note));
14  // 用于存储音符和时长的 ROM
15  song_rom song_rom_inst(.clk(clk), .dout({note[5:0], duration[5:0]}),
    .addr({song[1:0], q[4:0]}));
16  // 检查时长与歌曲是否完成
17  judge judge_inst(.clk(clk), .duration(duration), .co(co), .song_done(
    song_done));
18
19 endmodule
```

Listing 4: song_reader 部分代码

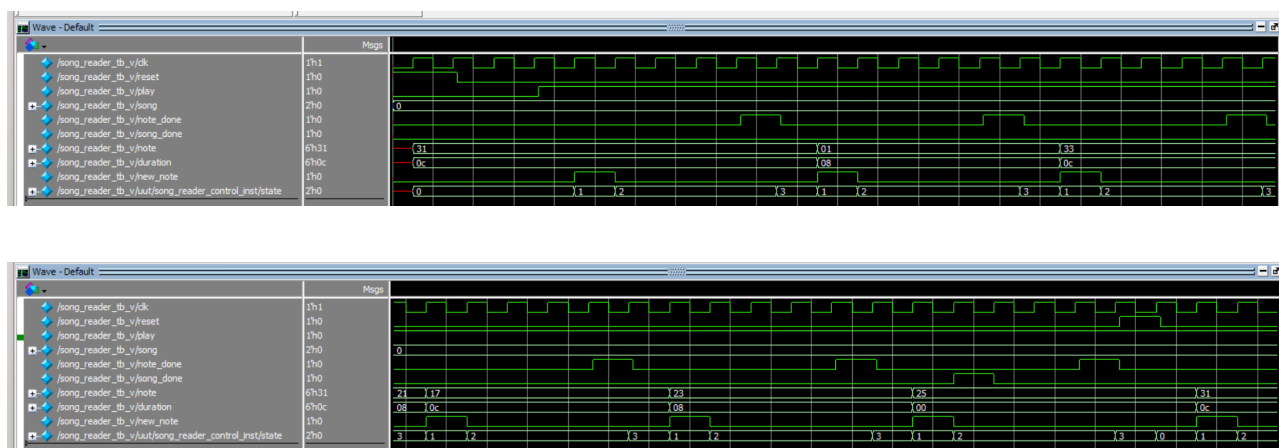
```

1  // 状态机逻辑
2  always @(*)
3  begin
4      new_note = 0;
5      case(state)
6          RESET: begin
7              if(play) nextstate = NEW_NOTE;
8              else nextstate = RESET;
9          end
10         NEW_NOTE: begin
11             new_note = 1;
12             nextstate = WAIT;
13         end
14         WAIT: begin
15             if(play != 1) nextstate = RESET;
16             else if(note_done) nextstate = NEXT_NOTE;
17             else nextstate = WAIT;
18         end
19         NEXT_NOTE: begin
20             nextstate = NEW_NOTE;
21         end
22         default: nextstate = RESET;
23     endcase
24 end

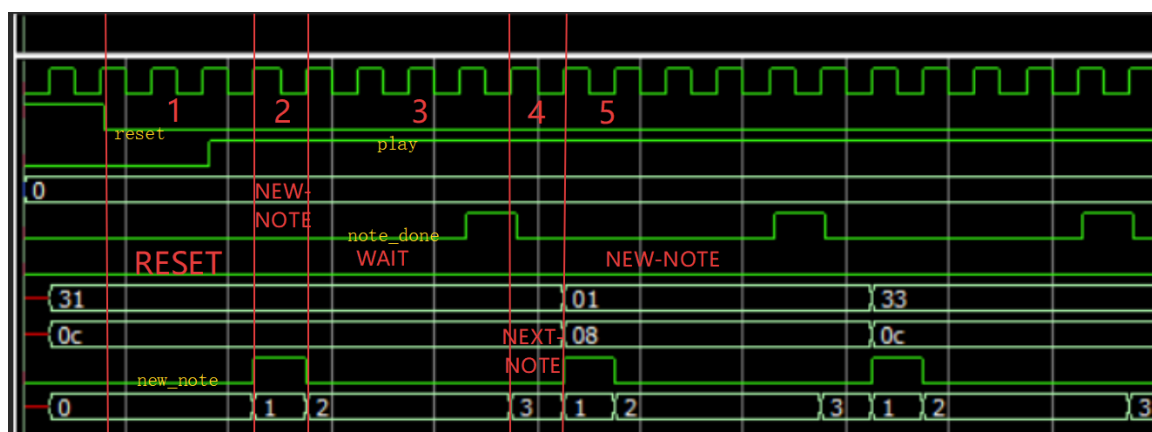
```

Listing 5: song_reader_control 核心代码

3.2 modelsim 仿真结果



放大：



区域 1 中，时钟边沿到来时 reset=1，进入 RESET 状态。

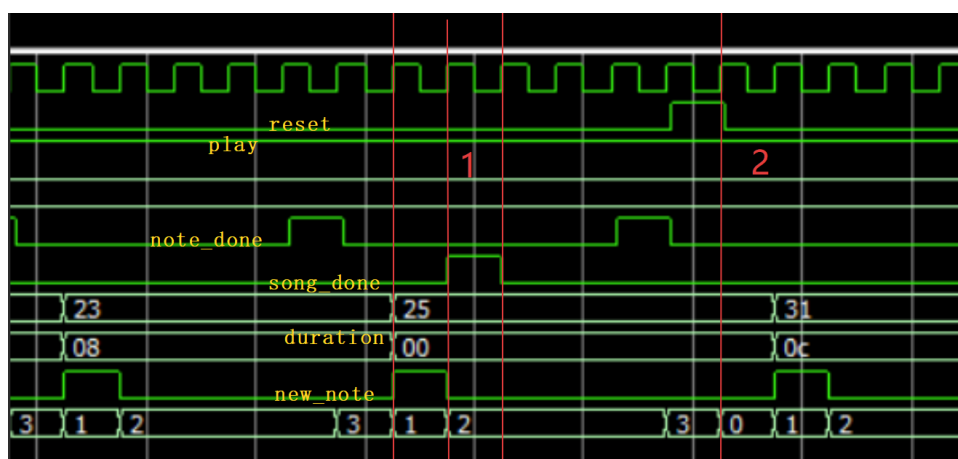
区域 2 中，时钟边沿到来时 play=1，进入 NEW_NOTE 状态，输出 new_note = 1，进行音符播放。

区域 3 中，note_done=0，进入 WAIT 状态。

区域 4 中，时钟边沿到来时 note_done=1，进入 NEXT_NOTE 状态，读取下一个音符的地址。

区域 5 中，时钟边沿到来时进入 NEW_NOTE 状态，输出 new_note=1。

结束放大：



区域 1 中，duration=0，输出一个时钟周期宽度的高电平脉冲信号 song_done，区域 2 中 reset=1，进入 RESET 状态。

4.note__player

4.1verilog 代码

核心代码:

```
1 module note_player(clk, reset, play_enable, note_to_load,
2   duration_to_load, load_new_note, note_done, sampling_pulse, beat,
3   sample, sample_ready);
4   input clk, reset, play_enable, load_new_note, sampling_pulse, beat;
5   input [5:0] note_to_load, duration_to_load;
6   output note_done, sample_ready;
7   output [15:0] sample;
8
9   // 用于保持当前音符的触发器
10  wire [5:0] qout;
11  wire load;
12  dffr #(.n(6)) dffr_inst(.r(~play_enable || reset), .en(load), .clk(
13    clk), .d(note_to_load), .q(qout));
14
15  // 用于获取音符频率值的ROM
16  wire [19:0] dout;
17  frequency_rom frequency_rom_inst(.clk(clk), .addr(qout), .dout(dout))
18    ;
19
20  // 用于生成样本的DDS
21  dds dds_inst(.clk(clk), .reset(~play_enable || reset), .
22    sampling_pulse(sampling_pulse), .k({2'b00, dout}), .sample(sample)
23    , .new_sample_ready(sample_ready));
24
25  // 音符播放器的控制单元
26  wire timer_clear, timer_done;
27  note_player_control note_player_control_inst(.clk(clk), .reset(reset)
28    , .play_enable(play_enable), .load_new_note(load_new_note), .
29    note_done(note_done), .load(load), .timer_clear(timer_clear), .
30    timer_done(timer_done));
31
32  // 用于跟踪音符时长的定时器
33  timer timer_inst(.clk(clk), .en(beat), .timer_clear(timer_clear), .
34    duration_to_load(duration_to_load), .q(), .timer_done(timer_done))
```

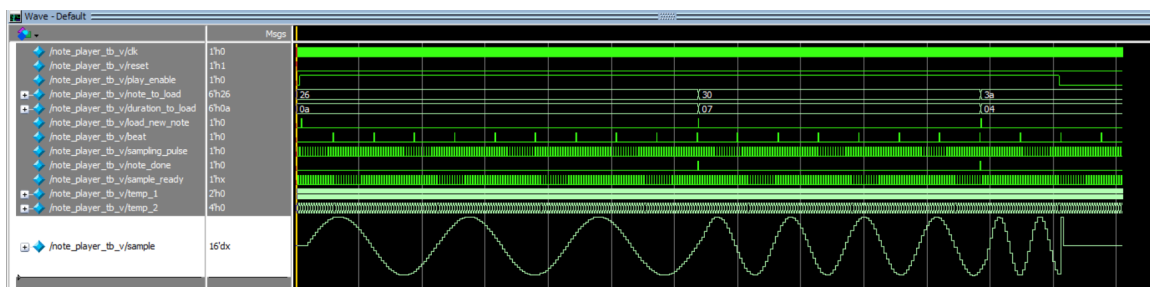
```
25 ;  
endmodule
```

Listing 6: note_player

```
1 always @(posedge clk)  
2     if(reset) state = RESET;  
3     else state = nextstate;  
4  
5 always @(*)  
6 begin  
7     timer_clear = 0; load = 0; note_done = 0;  
8     case (state)  
9         RESET: begin timer_clear = 1; nextstate = WAIT; end  
10        WAIT: begin  
11            if(!play_enable) nextstate = RESET;  
12            else if(timer_done) nextstate = DONE;  
13            else if(load_new_note) nextstate = LOAD;  
14            else nextstate = WAIT;  
15        end  
16        DONE: begin timer_clear = 1; note_done = 1; nextstate = WAIT;  
17            end  
18        LOAD: begin timer_clear = 1; load = 1; nextstate = WAIT; end  
19        default: nextstate = RESET;  
20    endcase  
21 end
```

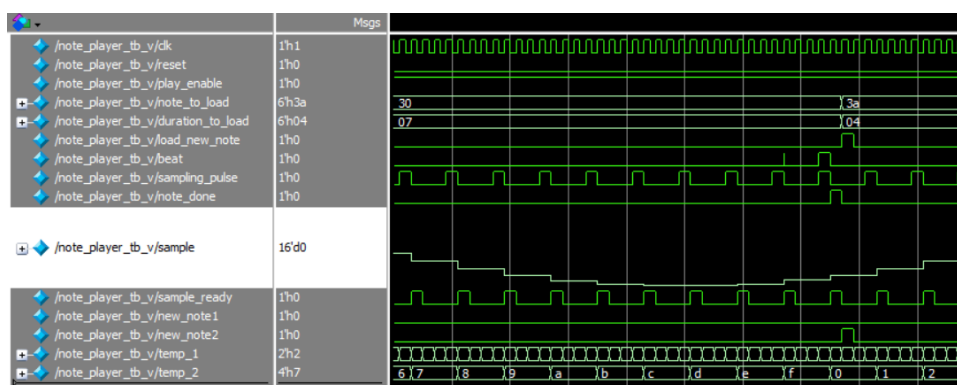
Listing 7: note_player_control 核心代码

4.2 modelsim 仿真结果



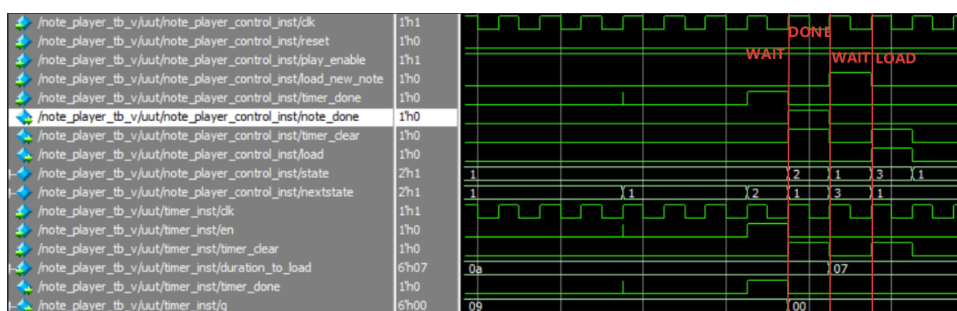
正弦信号频率与 `note_to_load` 的值正相关，当 `play_enable=0` 时，`sample=0`。

放大：



当 note_down 信号变为高电平，load_new_note 信号产生，更新播放音符. 模块设计成功。

控制器 note_player_control_inst：



当 note_player_ctrl 处于 WAIT 状态时，计时 q=0 时，输出 timer_done=1，持续 1 个时钟周期，控制器进入 DONE 状态，输出 timer_clear=1，note_done=1，进入 WAIT 状态，timer_clear=0，note_done=0，play_enable=1，timer_done=0，load_new_note=1，进入 LOAD 状态，输出 timer_clear=1，load=1。

则该模块的设计符合要求、

5.sync

核心代码：

```
1 module sync(in, clk, out);
2     input in, clk;
3     output out;
4
5     wire q1, q2;
6     assign out = q1 && ~q2;
```

```
7
8     dffr #(.n(1))dffr1(.r(0), .en(1), .clk(clk), .d(in), .q(q1));
9     dffr #(.n(1))dffr2(.r(0), .en(1), .clk(clk), .d(q1), .q(q2));
10 endmodule
```

Listing 8: sync

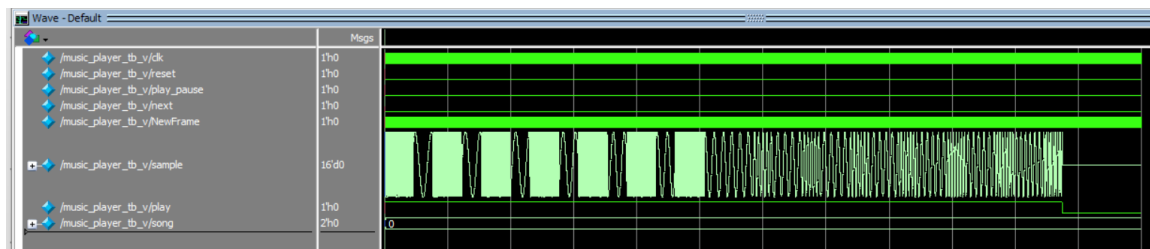
6.music_player

6.1verilog 代码

```
1 module music_player(clk, reset, play_pause, next, NewFrame, sample, play,
2     song);
3     parameter sim = 0;
4     input clk, reset, play_pause, next, NewFrame;
5     output [15:0] sample;
6     output [1:0] song;
7     output play;
8
9     wire [5:0] note, duration;
10    wire [1:0] song;
11    wire play, reset_play, song_done, ready, note_done, new_note, beat;
12
13    mcu mcu_inst(.clk(clk), .reset(reset), .play_pause(play_pause), .next
14        (next), .song_done(song_done), .song(song), .reset_play(reset_play
15        ), .play(play));
16    song_reader song_reader_inst(.clk(clk), .reset(reset_play), .
17        note_done(note_done), .play(play), .song(song), .song_done(
18        song_done), .new_note(new_note), .duration(duration), .note(note))
19        ;
20    note_player note_player_inst(.clk(clk), .reset(reset_play), .
21        play_enable(play), .note_to_load(note), .duration_to_load(duration
22        ), .load_new_note(new_note), .note_done(note_done), .
23        sampling_pulse(ready), .beat(beat), .sample(sample), .sample_ready
24        ());
25    sync sync_inst(.in(NewFrame), .clk(clk), .out(ready));
26    counter_n #(.n(sim?64:1000), .counter_bits(sim?6:10))counter_inst1(.
27        clk(clk), .en(ready), .r(0), .q(), .co(beat));
28 endmodule
```

Listing 9: music_player

6.2modelsim 仿真结果



波形符合预期，模块设计成功。

七、vivado 工程

新建 music_player 的 Vivado 工程, 编写顶层 music_player_top 模块的 Verilog HDL 代码, 生成符合要求的 DCM 内核, 添加 music_player 模块及其子模块、音频编解码接口模块的网表文件 AudioInterface.edf 和端口文件 AudioInterface.v、按键处理模块提供综合网表文件 button_press_unit.edf 和端口文件 button_press_unit.v 等。对工程进行综合、约束、实现, 并下载工程文件到 Nexys Video 开发板中。

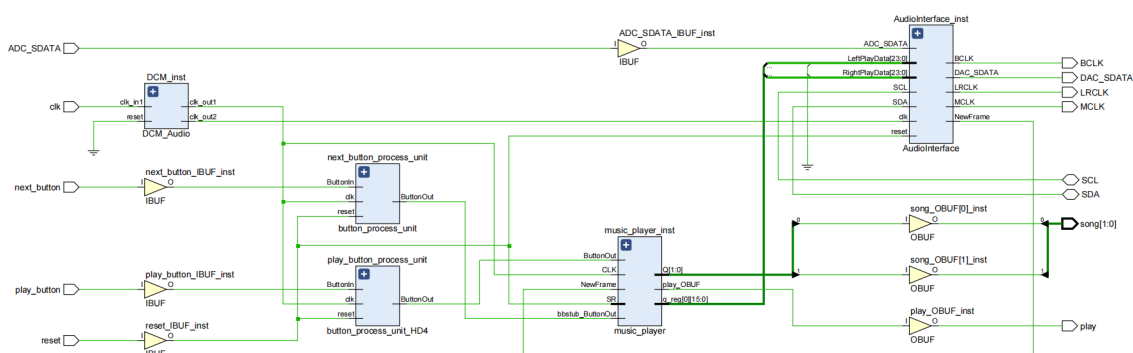


图 12: 原理图

将耳机接入实验开发板音频输出插座, 操作 reset(中间按钮)、play/pause(右边按钮)、next (下面按钮) 三个按键, 试听耳机中的乐曲并观察实验板上指示灯变化情况, 试听结果正常, 完成验收。

八、思考题

(1) 在实验中，为什么 next_button、play_pause_button 两个按键需要消颤动及同步化处理，reset 按键不需要消颤动及同步化处理？

reset 是置零信号，当 reset=1 时，系统置零，之后 reset 为 0 或为 1 系统内部该置零的信号都为 0，即 reset 的颤动对系统无影响。而 next_button 和 play_pause_button 都会影响系统内部状态的转换，如果 next 输入不稳定，则可能会跳到后面几首歌，也播放的下一首歌不确定；如果 play_pause 输入不确定，系统会不断地在播放和暂停之间切换，这不仅会增加系统的损耗，而且会输出断断续续的音乐，系统不稳定。

(2) 在主控制器 (mcu) 设计中，是否存在接收不到按键信息？若存在，概率多大？有必要修改设计？

存在。在 RESET 到 PAUSE 状态转换过程中和 NEXT 到 PLAY 状态转换过程中没有判断 next 和 play_pause，这期间按下按钮接收不到按键信息。概率很小，只有一个时钟周期，没有必要修改设计。

九、心得与体会

这次实验我用了一个多月完成，刚开始的时候几乎无从下手，当时数电理论课好像刚讲到时序电路吧，看到书上顶层框图中有那么多部分，我几乎头都大了。在仔细阅读书上的内容之后，我发现这个工程实际上是一个模块一个模块地写代码，在控制器中进行接线之类的。把一个大的项目分成许多小的模块进行调试，原来“吓人的”顶层框图似乎也不那么难了。工程起始阶段另外一个让我困扰的点就是我以为整个 sine_rom 都是要自己手工编写的，那工作量也太大了！后来看 DDS 部分的教材才发现正弦表可以直接用编程的方式生成，这也启示我要好好利用教材。这次实验也提高了我使用 Modelsim、Vivado 的熟练度，一开始的时候建立工程还需要翻书查看步骤，后来已经可以独立迅速地操作了。

整个设计过程中，我先根据 lab15 的内容完成 DDS 模块的设计，音乐播放器的设计中先把 song_reader、note_player、mcu 等子模块的接口定义清楚，再逐个实现内部逻辑。控制器的状态机设计是核心难点，我花了不少时间琢磨状态转换图与 Verilog 代码的对应关系。根据算法流程图写控制器的代码就有一些挑战，我主要参考了课本第二章“数字系统设计实例”部分的内容，花了一部分时间看懂状态转换的表达方式，再写起来就轻松一些了。小模

块 full_adder,counter_n 等在之前的实验中也有用到,在这个大工程中用得就比较顺手了。不过,结束判断模块在设计时还是费了一点时间。

仿真和 vivado 工程中,我也遇到了一些问题。首先是仿真波形不知道如何分析、结果对不对。后来认真地看了学在浙大上的调试视频,就能够添加内部信号、设置模拟波形显示等,根据波形的变化判断设计是不是对的。vivado 工程中也遇到了很多问题,主要是不够细心导致的。有忘记引脚约束导致 impementation 一直在初始化阶段的,还有两个网表文件没有添加到工程导致报错的,这些粗心的小错误提醒我一定要严谨细心地操作。上板调试时,还遇到了“灯亮但无声音”的奇怪现象。这个现象的原因其实令人啼笑皆非:是耳机接口接错了。我一开始还很慌张,以为设计又出了问题,后来问同学才知道是耳机的问题,这启示我遇到问题不要慌乱,要冷静地寻找原因。

总之,通过音乐播放器设计实验,我学到了很多,建立了数字系统设计的工程思维:自顶向下分解、模块化复用、状态机建模、仿真先行验证、硬件调试定位。这次经历将理论课上学到的触发器、加法器、计数器、状态机等知识真正“活”了起来,让我对数字电路有了从器件到系统的完整认知。从一开始的一团乱麻到现在完成了整个项目,还是很有成就感的。最后,非常感谢老师在设计过程中的耐心指导!